Lesson 14 Teacher's Key: Raising the Stakes

GameMaker - Instructor Reference Only

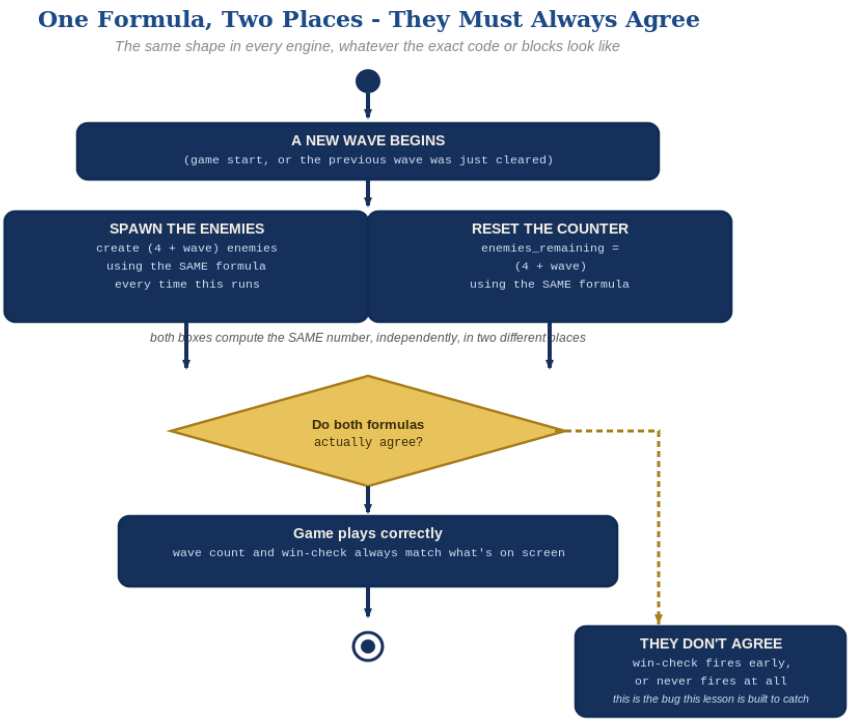
VIDEO GAME CREATION

What This Lesson Teaches

Same idea as every engine this week - replace hardcoded literals with wave-dependent formulas - expressed as GML, split across spawn_enemies() (its own Script resource since Lesson 11) and the two places obj_gamemanager.enemies_remaining already gets reset.

Level 1 - Full Worked Solution

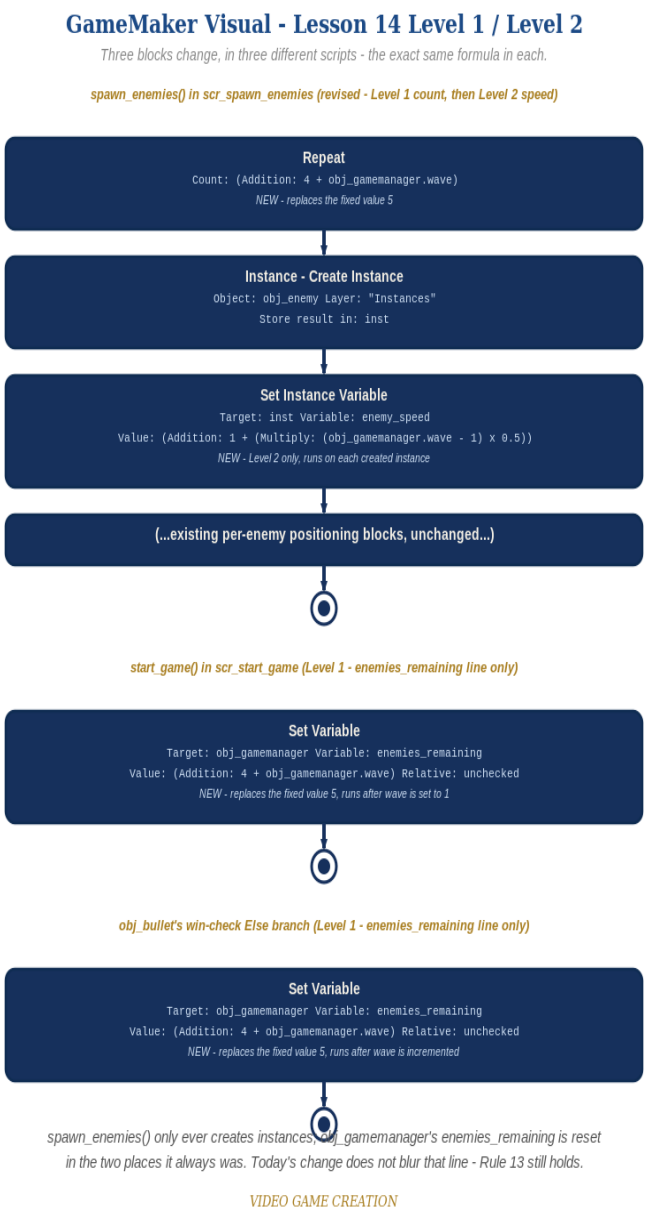
- repeat (4 + obj_gamemanager.wave) inside spawn_enemies() and enemies_remaining = 4 + wave inside both start_game() and obj_bullet's win-check else branch must all three agree - any mismatch is exactly the kind of bug this Key's troubleshooting section calls out.
- No new instance variables are needed for count scaling - only Level 2's speed scaling introduces one (enemy_speed on obj_enemy).
- Wave 1 still spawns exactly 5 (4 + 1 = 5), so nothing about the game's opening feel changes - only later waves escalate.



Whether it's two places (most engines) or the loop and the reset living in genuinely different scripts (Godot, GameMaker), the rule is the same: one formula, referenced everywhere.

VIDEO GAME CREATION

Whether the spawn loop and the reset live in two places or two genuinely different scripts, the rule is the same: one formula, referenced everywhere.



spawn_enemies(), start_game(), and obj_bullet's win-check Else branch - three blocks change, in three different scripts, the exact same formula in each. (the Visual path)

Why This Structure

- Keeping spawn_enemies()'s loop count and obj_gamemanager's enemies_remaining reset as separate concerns - exactly as Lesson 11 originally split them - means today's change does not blur that line even though both numbers now depend on the same wave: spawn_enemies() still only creates instances, obj_gamemanager still owns all shared state, per Rule 13.
- Setting enemy_speed on each freshly-created instance (inst.enemy_speed = ...) rather than changing obj_enemy's Create-event default keeps the scaling logic entirely inside spawn_enemies() - obj_enemy's Step event stays exactly as simple as Lesson 7 left it, just reading a variable instead of a literal.

Level 2 / Level 3 Talking Points

- Ask students to trace spawn_enemies() for wave 3 by hand: 4 + 3 = 7 - then separately confirm obj_gamemanager.enemies_remaining's reset computes the same 7 - a good comprehension check for a formula split across two places.
- For students comparing engines: point out that TIC-80's identical (4 + wave) formula, expressed as plain Lua instead of GML, is conceptually the exact same idea - a good cross-engine parallel, and both engines this week needed to introduce a NEW per-instance/per-entry speed value where none existed before.

Common Student Mistakes

- Updating spawn_enemies()'s repeat count but forgetting to update one (or both) of obj_gamemanager.enemies_remaining's two reset spots - or vice versa.
- Forgetting the obj_gamemanager. prefix on wave inside spawn_enemies() (it is a free-standing global function, not owned by obj_gamemanager) while correctly leaving it off inside code that already runs in obj_gamemanager's own context.
- Setting enemy_speed on the wrong instance (e.g. self instead of inst, or vice versa) inside spawn_enemies().
- Forgetting to change obj_enemy's Step event from "y += 1;" to "y += enemy_speed;" - the variable exists but is never actually read.